Diagnóstico del modelo relacional actual

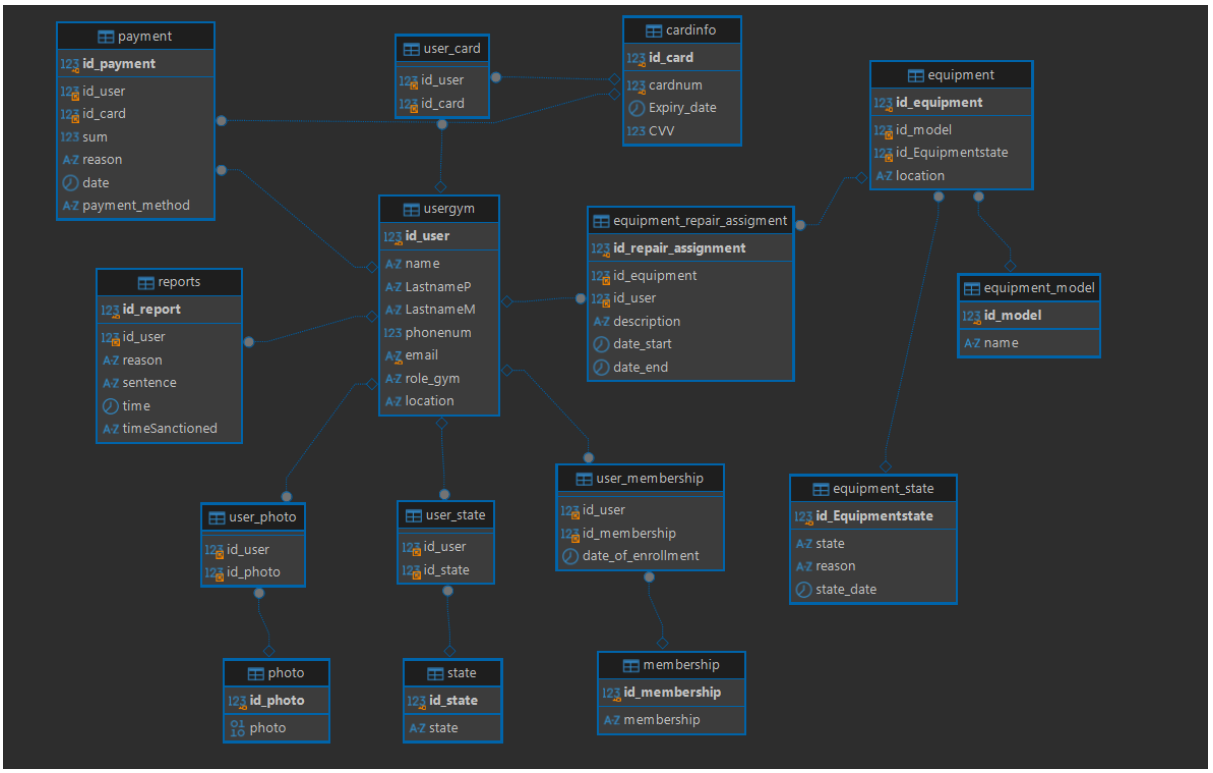


Figura 1

En la distribución actual se observan joins costosos ya que se presentan variedad de llaves foráneas. Por ejemplo, para obtener la información de pago y membresía de un usuario hay que unir las tablas de cardinfo con la de usergym mediante la de user_card y después la de membership con la de user_membership, haciendo que se acumulen las tablas en el join, aumentando el costo de generar la entidad de "Pago y membresía".

Por culpa del modelo relacional no se pueden poner elementos grandes y completos en un solo campo de una tabla, tienen que separarse en tablas y campos diferentes, formando un dato compuesto por estas. La tabla de equipment_repair_assignment, la cual consiste en ver las tareas registradas y el responsable de estas, depende de la tabla de usergym, equipment y las adyacentes a éstas para componer los datos totales de la tabla. A su vez tampoco se pueden guardar varios elementos en un solo campo por las mismas reglas de la estructura relacional.

En este modelo no existen jerarquías exactamente, se simulan roles en usergym para identificar el tipo de usuario que es, si fuera orientada a objetos pudiera heredar fácilmente el tipo y hasta poder tener implementado el funcionamiento de ese tipo desde un principio en el modelo y no en el programa.
Es necesario especificar y establecer un id para cada dato, haciéndolo mas tedioso, tardado y engorroso que los OID que se les asignan a los objetos en su modelo.

Aquí no se desarrolla el comportamiento de cada tabla y su manera de interactuar, solo es el almacenamiento de datos sin ninguna lógica detrás para funcionar junto a un sistema.

Estrategia de persistencia

Debido a que el modelo utilizado fue Durus, el modelo de persistencia será por alcanzabilidad, nativo de Durus. Debido a esto, los objetos no son marcados para saber cuales son guardados, sino son designados objetos raíz, y automáticamente los objetos que son alcanzados por esta misma raíz se vuelven persistentes gracias a la cadena de referencias que forman entre sí, entonces el conjunto guardado es todo el ciclo de alcanzabilidad desde la raíz. Esta estrategia concuerda bien con nuestra estructura, ya que al tener información dependiente de otra, como sería un cliente, su método de pago y membresía, todo persiste automáticamente y de manera consistente.

La raíz es el objeto connection.get_root() llamado “raíz”, la cual servirá como diccionario para encontrar las colecciones de lo que almacenamos, donde el PersistentDict almacenará el contenido de estas colecciones por sus etiquetas.

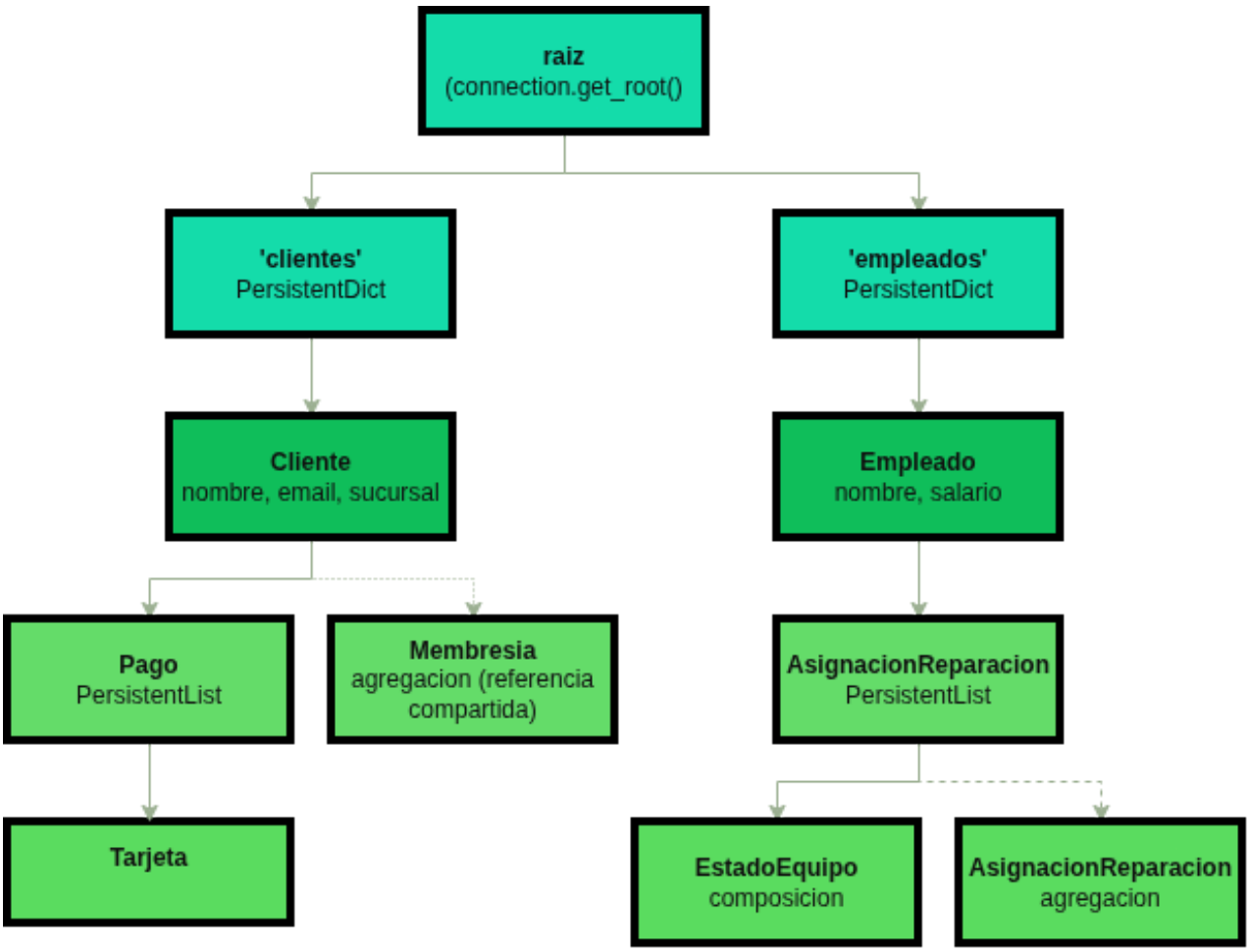


Figura 3

Definición formal del esquema

```
module Gym {

    interface User {
        attribute string name;
        attribute string lastnameP;
        attribute string lastnameM;
        attribute string phoneNum;
        attribute string email;
        void updateProfile();
    };

    interface Client : User {
        attribute date enrollmentDate;
        relationship List<Payment> paymentHistory
            inverse Payment::client;
        relationship Membership membership
            inverse Membership::clients;
        void pay(in float amount);
```

```
        void enroll();
    };

    interface Payment {
        attribute float amount;
        attribute string method;
        attribute date date;
        relationship Client client
            inverse Client::paymentHistory;
        relationship Card card
            inverse Card::payments;
    };

};
```

Consultas OQL ejemplo

1. OQL:

```
SELECT c.name
FROM Clients c
WHERE c.membership.type = ‘plus’
```

en SQL:

```
SELECT u.name
FROM usergym u
JOIN user_membership um ON u.id_user = um.id_user
JOIN membership m ON um.id_membership = m.id_membership
WHERE m.membership = 'plus';
```

2. OQL

```
SELECT p.amount, p.card.cardNum
FROM Clients c, c.paymentHistory p
WHERE p.amount > 500;
```

en SQL

```
SELECT p.sum, c.cardnum
FROM Payment p
JOIN cardinfo c ON p.id_card = c.id_card
WHERE p.sum > 500;
```

Graph navigation

```
cliente = raiz['usuarios']['ana@email.com']
for pago in cliente.historial_pagos:
    print(f"Pago de ${pago.monto}")
```

El punto de entrada es siempre la raiz, donde entra a la coleccion de clientes y extrae el cliente que tiene de clave el correo para entregar el objeto “Ana”, de ahi se va a la PersistentList que tiene el cliente de pagos y la recorre pago por pago.

```
for pago in cliente.historial_pagos:
    print(f"Tarjeta: {pago.tarjeta.num_tarjeta}")
```

vuelve a recorrer los pagos del cliente uno por uno, despues entra hacia la referencia del pago de esa tarjeta, y de ahi se lee el atributo del numero de tarjeta.